SIMPLY FPU

by **Raymond Filiatreault**

Chap. 6
Data transfer instructions - Packed decimals

The FPU instructions covered in this chapter perform no mathematical operation on numerical data. Their main purpose is simply to transfer packed Binary Coded Decimal (BCD) integer data between the FPU's 80-bit data registers and memory.

There are only two FPU instructions using packed BCD data as the operand and no other data type can be used with those instructions. Both have the letter "B" immediately following the "F" in the mnemonic.

FBLD LoaD BCD data from memory

FBSTP STore BCD data to memory

FBLD (Load BCD data from memory)

Syntax: **fbld *Src***

Exception flags: Stack Fault, Invalid operation

This instruction decrements the TOP register pointer in the Status Word and loads the packed BCD integer value from the specified source (*Src*) in the new TOP data register ST(0) after converting it to the 80-bit REAL10 format. The source must be the memory address of an 80-bit TBYTE having the specified packed BCD format. (See Chap.2 for the packed BCD format and its addressing modes).

If the ST(7) data register which would become the new ST(0) is not empty, both a **S**tack **F**ault and an **I**nvalid operation exceptions are detected, setting both flags in the Status Word. The TOP register pointer in the Status Word would still be decremented and the new value in ST(0) would be the INDEFINITE NAN.

The content of the Tag Word for the new TOP register will be modified according to the result of the operation.

Although each nibble (4 bits) in a proper packed BCD format should not exceed a value of 9, the FPU will convert any value without any exception being detected. If a nibble does exceed a value of 9, the FPU simply uses its mod 10 value and carries a 1 to the next nibble. For example,

if a TBYTE contains the hex value of **0000000000000009A7Dh**,

the FBLD instruction would convert it as the decimal value of **10083d**

or the same as a proper packed BCD value of **0000000000000010083h**.

The FBLD instruction is used primarily for converting alphanumeric strings (which usually contain a fractional part) to the REAL10 format. The process involves

- parsing the string for invalid characters to avoid processing "garbage",
- determining the number of decimal places,
- converting the digits to the packed BCD format,
- loading the formatted number to the FPU as an integer, and
- dividing by the appropriate exponent of 10 according to the number of decimal places in the original string.

Following is an example of code to convert a decimal string to the packed BCD format. This example would only allow a string in the regular decimal notation. Separate instructions would be needed to also accept a string in the scientific notation. (Code is not included for the final steps of loading it to the FPU and dividing by an exponent of 10 to arrive at the final floating point value.)

****

//The packed BCD value will be stored in a global memory variable. A local variable or some other memory location could be used as well for this purpose. The same applies for the location where the null-terminated "user input" string will be available.

This code will return the packed BCD value in memory, the total number of integer/decimal digits in AH and the number of decimal digits in AL. If an error is detected (no input digit, more than 18 digits, invalid character), EAX will return with 0, and ECX will contain the counts of digits.//

```
.data

temp_bcd     dt   ?
szdecimal    db   32 dup(0)

.code

a2bcd proc

; EDI will be used as the pointer for the location of the BCD value
; ESI will be used as the pointer to the source decimal string
; ECX will be used to hold the count of integer digits (in CH)
; and decimal digits (in CL)

   push esi
   push edi
   xor  ecx,ecx         ;initialize for the counts
   lea  esi,szdecimal-1 ;point to byte preceding the source string
   lea  edi,temp_bcd    ;point to the memory location for the packed BCD

   xor  eax,eax
   stosd
   stosd
   stosw                ;initializes the TBYTE to 0, including the sign byte
   dec  edi             ;adjust for pointing to the sign byte
                        ; (most significant byte)
                        ;which now assumes the value as positive

; The following is to skip leading spaces without generating an error

space_check:
   inc  esi             ;point to next character
   mov  al,[esi]        ;get character
   cmp  al," "          ;is it a space
   jz   space_check     ;check for another space

; Check 1st non-space character for + or - sign

   .if  al == "-"       ;is there a "-" sign
      mov  byte ptr [edi],80h ;changes the sign byte for the negative code
      inc  esi          ;point to next character

   .elseif al == "+"    ;is there a "+" sign
      inc  esi          ;point to next character,
                        ; the sign byte is already coded positive
   .endif

integer_count:
   .if  al == 0         ;is it the end of the source string
      jmp count_end
   .elseif al == "."    ;is it a "period" decimal delimiter
      jmp decimal_count
```

```
    .elseif al == ","    ;is it a "coma" decimal delimiter
                         ; (also used in numerous countries)
       jmp decimal_count
    .elseif al < "0"     ;is it an invalid character below the ASCII 0
       jmp input_error
    .elseif al > "9"     ;is it an invalid character above the ASCII 9
       jmp input_error
    .elseif ch == 18     ;is digit count already at the maximum allowed
       jmp input_error
    .endif

    inc  ch              ;increment the count of integers digits
    inc  esi             ;point to next character
    mov  al,[esi]        ;get character
    jmp  integer_count   ;continue counting integer digits

 decimal_count:
    inc  esi             ;point to next character
    mov  al,[esi]        ;get character
    .if  al == 0         ;is it the end of the source string
       jmp count_end
    .elseif al < "0"     ;is it an invalid character below the ASCII 0
       jmp input_error
    .elseif al > "9"     ;is it an invalid character above the ASCII 9
       jmp input_error
    .elseif ch == 18     ;is digit count already at the maximum allowed
       jmp input_error
    .endif

    inc  cl              ;increment the count of decimals digits
    inc  ch              ;increment the count of integers+decimals digits
    jmp  decimal_count   ;continue counting decimal digits

 count_end:
    .if  ch == 0         ;check if input has any digit
       jmp input_error
    .endif

 ; At this point, the input string is valid,
 ; CL contains the count of decimal digits,
 ; CH contains the total count of decimal+integer digits, and
 ; ESI points to the terminating 0.
 ; All the digits now have to be converted to binary and
 ; packed two per byte from the least significant to the more significant

    push ecx             ;for temporary storage of the counts
    sub  ch,cl           ;CH now contains the count of only the integer digits
    lea  edi,temp_bcd    ;point to the least significant byte of the packed BCD

 pack_decimals:
    dec  esi             ;point to the next more significant digit
    mov  al,[esi]        ;get decimal character

    .if  cl == 0         ;check if decimal digits all done
       .if  al < "0"     ;checks if current character is the decimal delimiter
          dec  esi       ;skip the decimal delimiter
       .endif
       jmp  pack_integers
    .endif

    and  al,0fh          ;keep only the binary portion
    ror  ax,4            ;transfer the 4 bits to the H.O. nibble of AH
    dec  esi             ;point to the next more significant digit
    dec  cl              ;decrement counter of decimal characters

    .if  cl == 0         ;check if decimal digits all done
       dec  esi          ;skip the decimal delimiter
       jmp  pack_integer2 ;get an integer character as 2nd nibble of this byte
    .endif
```

```
   mov  al,[esi]       ;get decimal character
   and  al,0fh         ;keep only the binary portion
   rol  ax,4           ;moves the 2nd binary value to the H.O. nibble of AL
                       ;and retrieves the 1st binary value from AH
                       ; into the L.O. nibble of AL
   stosb               ;store this packed BCD byte and increment EDI
   dec  cl             ;decrement counter of decimal characters
   jmp  pack_decimals  ;continue packing the decimal digits

pack_integers:
   .if  ch == 0        ;check if integer digits all done
      jmp  all_done    ;and terminate the packing process
   .endif

   mov  al,[esi]       ;get integer character
   and  al,0fh         ;keep only the binary portion
   ror  ax,4           ;transfer the 4 bits to the H.O. nibble of AH
   dec  esi            ;point to the next more significant digit
   dec  ch             ;decrement counter of integer characters

pack_integer2:
   .if  ch == 0        ;check if integer digits all done
      rol  ax,4        ;moves the 2nd binary value to the H.O. nibble of AL
                       ;and retrieves the 1st binary value from AH
                       ; into the L.O. nibble of AL
      mov  [edi],al    ;store this packed BCD byte
      jmp  all_done    ;and terminate the packing process
   .endif

   mov  al,[esi]       ;get integer character
   and  al,0fh         ;keep only the binary portion
   rol  ax,4           ;moves the 2nd binary value to the H.O. nibble of AL
                       ;and retrieves the 1st binary value from AH
                       ; into the L.O. nibble of AL
   stosb               ;store this packed BCD byte and increment EDI
   dec  esi            ;point to the next more significant digit
   dec  ch             ;decrement counter of integer characters
   jmp  pack_integers  ;continue packing the integer digits

all_done:
   pop  eax            ;retrieve the counts which were in ECX
   pop  edi
   pop  esi
   ret

input_error:
   xor  eax,eax        ;to return with error code
   pop  edi
   pop  esi
   ret

a2bcd endp
```

****

FBSTP (Store BCD data to memory)

Syntax: **fbstp *Dest***

Exception flags: Stack Fault, Invalid operation, Precision

This instruction rounds the value of the TOP data register ST(0) to the nearest integer (regardless of the rounding mode of the RC field in the Control Word), converts it to the packed BCD format, and stores it at the specified destination (*Dest*). (See Chap.2 for the packed BCD format and its addressing modes). It then

POPs the TOP data register ST(0), modifying the Tag Word and incrementing the TOP register pointer of the Status Word.

> *If the floating point value needs to be rounded according to some other mode, the FRNDINT instruction must be used prior to the FBSTP instruction.*

If the ST(0) data register is empty, both a **S**tack **F**ault and an **I**nvalid operation exceptions are detected, setting both flags in the Status Word. The INDEFINITE value in REAL10 format (FFFFC000000000000000h) would be stored as such at the specified destination.

If the rounded value of ST(0) would contain more than 18 integer digits, an **I**nvalid operation exception is detected, setting the related flag in the Status Word. The INDEFINITE value in REAL10 format would be stored as such at the specified destination.

If the value in ST(0) would contain a binary fraction, some precision would be lost and a **P**recision exception would be detected, setting the related flag in the Status Word.

The FBSTP instruction is used primarily for converting floating point values from ST(0) to alphanumeric strings (usually with a required number of decimal places). The process involves:
- multiplying the value in ST(0) by an appropriate exponent of 10 according to the number of required decimal places, (taking precautions not to exceed 18 significant decimal digits in the resulting integer portion),
- storing it to a memory location as a packed BCD format,
- converting from the packed BCD format to a null-terminated alphanumeric string.

> If the value to be converted is $\geq 10^{19}$ or <1, it should generally be converted to the scientific notation. The exception may be for small values close to 1 which need to be reported only to a few decimal places.

Following is an example of code to convert from the packed BCD format to a decimal string in regular notation (some modification would be needed to prepare a decimal string in scientific notation).

****

//The packed BCD value will be stored in a global memory variable. A local variable or some other memory location could be used as well for this purpose. The same applies for the location where the resulting null-terminated decimal string will be stored.

This code assumes that the floating point value is currently in the FPU's ST(0) register. It also assumes that 2 decimal places are required and that the resulting decimal value will not contain more than 18 digits. No error checking is performed.

The decimal string will also be left-justified. A "-" sign will be inserted as the first character if the value is negative, but nothing is inserted if the value is positive. Those are all additional options for the programmer.//

.data

```
temp_bcd     dt   ?
szdecimal    db   32 dup(0)

.code

bcd2a PROC

   pushd 100            ;use the stack as storage for this value
   fimul dword ptr[esp] ;converts 2 decimal places as 2 more integer digits
   fbstp temp_bcd       ;store the packed BCD integer value
   pop  eax             ;clean the stack of the pushed 100

; ESI will be used for pointing to the packed BCD
```

```
; EDI will be used for pointing to the decimal string

   push esi
   push edi
   lea  esi,temp_bcd+9  ;point to the most significant byte
   lea  edi,szdecimal   ;point to the start of the decimal string buffer
   xor  eax,eax
   fwait                ;to ascertain that the transfer of the
                        ;packed BCD is completed

   mov  al,[esi]        ;get the byte with the sign code
   dec  esi             ;decrement to next most significant byte
   or   al,al           ;for checking the sign bit
   jns  @F              ;jump if no sign bit
   mov  al,"-"          ;the value is negative
   stosb                ;insert the negative sign

@@:

; The next 8 bytes (in this example) will contain the integer digits
; and the least significant byte will then contain the 2 decimal digits.
; No leading 0 will be included in the integer portion
; unless it would be the only integer digit.

   mov  ecx,8           ;number of bytes to process for integer digits

@@:

   mov  al,[esi]        ;get the next byte
   dec  esi             ;adjust the pointer to the next one
   or   al,al           ;for checking if it is 0
   jnz  @F              ;the starting integer digit is now in AL
   dec  ecx             ;adjust the counter of integer bytes
   jnz  @B              ;continue searching for the first integer digit

; If the loop terminates with ECX=0, the integer portion would be 0.
; A "0" character must be inserted before processing the decimal digits

   mov  al,"0"          ;the ASCII 0
   stosb                ;insert it
   mov  al,[esi]        ;get the byte containing the decimal digits
   jmp  decimal_digits

@@:

; The first integer byte must be checked to determine
; if it contains 1 or 2 integer digits

   test al,0f0h         ;test if the H.O. nibble contains a digit
   jz   int_digit2      ;if not, process only the L.O. nibble

int_digit1:
   ror  ax,4            ;puts the H.O. nibble in the L.O. nibble position
                        ;and saves the L.O. nibble in AH
   add  al,30h          ;convert it to ASCII character
   stosb                ;store this character
   shr  ax,12           ;restores the L.O. nibble in AL
                        ;and also resets the other bits of AX

int_digit2:
   add  al,30h          ;convert it to ASCII character
   stosb                ;store this character
   mov  al,[esi]        ;get next byte
   dec  esi             ;adjust the pointer to the next one
   dec  ecx             ;adjust the counter of integer bytes
   jnz  int_digit1      ;continue processing the integer bytes

decimal_digits:
   mov  byte ptr [edi],"." ;insert the preferred decimal delimiter
```

```
    inc  edi

; If more than 2 decimal digits are required, a counter would be needed
; to process the necessary bytes.
; Also, if the number of required decimal digits is not even, the code
; would have to be altered to insert the decimal delimiter at the
; proper location.

    ror  ax,4           ;puts the H.O. nibble in the L.O. nibble position
                        ;and saves the L.O. nibble in AH
    add  al,30h         ;convert it to ASCII character
    stosb               ;store this character
    shr  ax,12          ;restores the L.O. nibble in AL
                        ;and also resets the other bits of AX
    add  al,30h         ;convert it to ASCII character
    stosw               ;store this character + the terminating 0

    pop edi
    pop esi
    ret

bcd2a ENDP
```

****

RETURN TO
SIMPLY FPU
CONTENTS